CSA 1737 – ARTIFICIAL INTELLIGENCE (Prolog)

Laboratory Record – Experiments 26 to 30

HOW TO RUN THE PROLOG PROGRAMS IN GOOGLE COLAB

Google Colab does not have SWI-Prolog by default. Install it once, then write each program into a file with %%writefile and run it with swipl. Each program ends with ":- initialization(main)." so the queries run automatically.

Cell 1 (run once):

```
!apt-get install -y -qq swi-prolog > /dev/null
```

Cell 2 (for every experiment; the first line must be %%writefile):

```
%%writefile ex26.pl
<paste the Prolog program here>
```

Cell 3 (run the program):

```
!swipl -q -s ex26.pl -t halt
```

Ex. No. 26: Sum of the Integers from 1 to n

AIM

To write a Prolog program to find the sum of the integers from 1 to n.

ALGORITHM

Step 1: Start the program.

Step 2: Base case: the sum of the integers up to 0 is 0.

Step 3: Recursive case: for N > 0, find the sum up to N-1 and add N to it.

Step 4: Query sum(N, S) to get the sum for a given N.

Step 5: Stop the program.

CODE

```
% Experiment 26: Sum of integers from 1 to n
sum(0, 0).
sum(N, S) :-
    N > 0,
    N1 is N - 1,
    sum(N1, S1),
    S is S1 + N.

% ---- Queries ----
query(N) :-
    sum(N, S),
    format("?- sum(~w, S).~nS = ~w~n~n", [N, S]).

main :-
    query(5),
    query(10),
    query(100).

:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex26.pl -t halt
```

OUTPUT

```
?- sum(5, S).
S = 15

?- sum(10, S).
S = 55

?- sum(100, S).
S = 5050
```

RESULT

Thus the Prolog program to find the sum of the integers from 1 to n was written and executed successfully, and the output was verified.

Ex. No. 27: Database with Name and DOB

AIM

To write a Prolog program to create a database of names and dates of birth and query it.

ALGORITHM

Step 1: Start the program.

Step 2: Create facts of the form person(Name, date(Day, Month, Year)).

Step 3: Define rules dob/2, born_in_year/2 and born_in_month/2.

Step 4: Run queries to find the date of birth of a person and the persons born in a given year or month.

Step 5: Display all the records using forall/2.

Step 6: Stop the program.

CODE

```
% Experiment 27: Database with NAME and DOB
% person(Name, date(Day, Month, Year))
person(ravi,   date(12, 5, 2003)).
person(priya,  date(23, 8, 2004)).
person(arun,   date(7, 1, 2003)).
person(meena,  date(30, 11, 2004)).
person(kiran,  date(15, 5, 2005)).

dob(Name, D/M/Y) :- person(Name, date(D, M, Y)).
born_in_year(Name, Y) :- person(Name, date(_, _, Y)).
born_in_month(Name, M) :- person(Name, date(_, M, _)).

% ---- Queries ----
main :-
    format("?- dob(priya, DOB).~n"),
    dob(priya, DOB), format("DOB = ~w~n~n", [DOB]),

    format("?- born_in_year(X, 2004).~n"),
    findall(X, born_in_year(X, 2004), L1), format("X = ~w~n~n", [L1]),

    format("?- born_in_month(X, 5).~n"),
    findall(X, born_in_month(X, 5), L2), format("X = ~w~n~n", [L2]),

    format("?- person(Name, Date).  (all records)~n"),
    forall(person(Name, date(D, M, Y)),
           format("Name = ~w, DOB = ~w-~w-~w~n", [Name, D, M, Y])).

:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex27.pl -t halt
```

OUTPUT

```
?- dob(priya, DOB).
DOB = 23/8/2004
```

```
?- born_in_year(X, 2004).
X = [priya,meena]

?- born_in_month(X, 5).
X = [ravi,kiran]

?- person(Name, Date).  (all records)
Name = ravi, DOB = 12-5-2003
Name = priya, DOB = 23-8-2004
Name = arun, DOB = 7-1-2003
Name = meena, DOB = 30-11-2004
Name = kiran, DOB = 15-5-2005
```

RESULT

Thus the Prolog program for the Name and DOB database was written and executed successfully, and the queries gave the expected answers.

Ex. No. 28: Student - Teacher - Subject Code Database

AIM

To write a Prolog program to create a database relating students, teachers and subject codes, and query it.

ALGORITHM

Step 1: Start the program.

Step 2: Create facts for subject(Code, Name), teaches(Teacher, Code) and studies(Student, Code).

Step 3: Define rules student_teacher/2, teacher_subject/2 and student_subject/2 that join these facts using the subject code.

Step 4: Run queries such as: who teaches a student, which students study a subject, and which subjects a student studies.

Step 5: Stop the program.

CODE

```
% Experiment 28: STUDENT - TEACHER - SUBJECT CODE
subject(csa17,   'Artificial Intelligence').
subject(csa04,   'Operating Systems').
subject(csa1064, 'Database Management').

teaches(kumar, csa17).
teaches(devi,  csa04).
teaches(raj,   csa1064).

studies(arun,  csa17).
studies(bala,  csa17).
studies(chitra, csa04).
studies(arun,  csa04).
studies(deepa, csa1064).

student_teacher(S, T)   :- studies(S, C), teaches(T, C).
teacher_subject(T, Sub) :- teaches(T, C), subject(C, Sub).
student_subject(S, Sub) :- studies(S, C), subject(C, Sub).

main :-
    format("?- subject(csa17, Name).~n"),
    subject(csa17, N), format("Name = ~w~n~n", [N]),

    format("?- student_teacher(arun, T).~n"),
    findall(T, student_teacher(arun, T), L1), format("T = ~w~n~n", [L1]),

    format("?- studies(S, csa17).~n"),
    findall(S, studies(S, csa17), L2), format("S = ~w~n~n", [L2]),

    format("?- teacher_subject(devi, Sub).~n"),
    teacher_subject(devi, Sub), format("Sub = ~w~n~n", [Sub]),

    format("?- student_subject(arun, Sub).~n"),
    findall(Sub2, student_subject(arun, Sub2), L3), format("Sub = ~w~n", [L3]).
```

```
:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex28.pl -t halt
```

OUTPUT

```
?- subject(csa17, Name).
Name = Artificial Intelligence

?- student_teacher(arun, T).
T = [kumar,devi]

?- studies(S, csa17).
S = [arun,bala]

?- teacher_subject(devi, Sub).
Sub = Operating Systems

?- student_subject(arun, Sub).
Sub = [Artificial Intelligence,Operating Systems]
```

RESULT

Thus the Prolog program for the Student-Teacher-Subject Code database was written and executed successfully, and the queries gave the expected answers.

Ex. No. 29: Planets Database

AIM

To write a Prolog program to create a database of the planets of the solar system and query it.

ALGORITHM

Step 1: Start the program.

Step 2: Create facts of the form planet(Name, Position, Diameter_km, Type) for the eight planets.

Step 3: Define the rule larger_than(X, Y) that compares the diameters of two planets.

Step 4: Run queries to find the planet at a given position, the terrestrial planets, the gas giants and the planets smaller than Earth.

Step 5: Stop the program.

CODE

```
% Experiment 29: PLANETS database
% planet(Name, Position, Diameter_km, Type)
planet(mercury, 1,   4879, terrestrial).
planet(venus,   2,  12104, terrestrial).
planet(earth,   3,  12742, terrestrial).
planet(mars,    4,   6779, terrestrial).
planet(jupiter, 5, 139820, gas_giant).
planet(saturn,  6, 116460, gas_giant).
planet(uranus,  7,  50724, ice_giant).
planet(neptune, 8,  49244, ice_giant).

larger_than(X, Y) :- planet(X, _, D1, _), planet(Y, _, D2, _), D1 > D2.

main :-
    format("?- planet(P, 3, _, _).~n"),
    planet(P, 3, _, _), format("P = ~w~n~n", [P]),

    format("?- planet(P, _, _, terrestrial).~n"),
    findall(P1, planet(P1, _, _, terrestrial), L1), format("P = ~w~n~n", [L1]),

    format("?- planet(P, _, _, gas_giant).~n"),
    findall(P2, planet(P2, _, _, gas_giant), L2), format("P = ~w~n~n", [L2]),

    format("?- planet(earth, Pos, Dia, Type).~n"),
    planet(earth, Pos, Dia, Type),
    format("Pos = ~w, Dia = ~w, Type = ~w~n~n", [Pos, Dia, Type]),

    format("?- larger_than(P, jupiter).~n"),
    ( findall(P3, larger_than(P3, jupiter), L3), L3 \== []
      -> format("P = ~w~n~n", [L3]) ; format("false.~n~n") ),

    format("?- larger_than(earth, P).~n"),
    findall(P4, larger_than(earth, P4), L4), format("P = ~w~n", [L4]).

:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex29.pl -t halt
```

OUTPUT

```
?- planet(P, 3, _, _).
P = earth

?- planet(P, _, _, terrestrial).
P = [mercury,venus,earth,mars]

?- planet(P, _, _, gas_giant).
P = [jupiter,saturn]

?- planet(earth, Pos, Dia, Type).
Pos = 3, Dia = 12742, Type = terrestrial

?- larger_than(P, jupiter).
false.

?- larger_than(earth, P).
P = [mercury,venus,mars]
```

RESULT

Thus the Prolog program for the Planets database was written and executed successfully, and the queries gave the expected answers.

Ex. No. 30: Towers of Hanoi

AIM

To write a Prolog program to implement the Towers of Hanoi problem.

ALGORITHM

Step 1: Start the program.

Step 2: Base case: if the number of disks is 0, do nothing.

Step 3: Move N-1 disks from the source peg to the helper (via) peg.

Step 4: Move the largest disk from the source peg to the destination peg and print the move.

Step 5: Move the N-1 disks from the helper peg to the destination peg.

Step 6: The minimum number of moves is 2^N - 1.

Step 7: Stop the program.

CODE

```
% Experiment 30: Towers of Hanoi
hanoi(0, _, _, _) :- !.
hanoi(N, From, To, Via) :-
    N1 is N - 1,
    hanoi(N1, From, Via, To),
    format("Move disk ~w from ~w to ~w~n", [N, From, To]),
    hanoi(N1, Via, To, From).

moves(N, M) :- M is 2 ** N - 1.

main :-
    N = 3,
    format("?- hanoi(~w, left, right, center).~n", [N]),
    hanoi(N, left, right, center),
    moves(N, M),
    format("~nMinimum number of moves for ~w disks = ~w~n", [N, M]).

:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex30.pl -t halt
```

OUTPUT

```
?- hanoi(3, left, right, center).
Move disk 1 from left to right
Move disk 2 from left to center
Move disk 1 from right to center
Move disk 3 from left to right
Move disk 1 from center to left
Move disk 2 from center to right
Move disk 1 from left to right

Minimum number of moves for 3 disks = 7
```

RESULT

Thus the Prolog program for the Towers of Hanoi was written and executed successfully; 3 disks were moved in 7 moves.